



**INSTITUTO DE EDUCACIÓN SECUNDARIA LA MOLA
NOVELDA**

DESARROLLO DE APLICACIONES WEB

Curso Académico 2025/2026

Desarrollo Web en Entorno Cliente

**U02_A06 Casos prácticos, Ponte a prueba, etc... -
Prof. Emilio Ramírez**

Autor: Segura Abad, Mario

Fecha: 31 de Octubre, 2025

ÍNDICE

INTRODUCCIÓN	3
ENUNCIADO	4
Caso práctico 6	5
Comprobación del modo estricto	5
Ponte a prueba 6	7
Enunciado	7
Claves de resolución:.....	8
DESARROLLO DE LA PRACTICA.....	9
Descripción	9
Contenido del archivo	9
Código del Apartado 6 de teoría	9
Caso práctico 6 – Comprobación del modo estricto	10
Ponte a prueba – Control de errores con objetos	11
Cómo visualizarlo	12
Requisitos:	12
Pasos:	12
Objetivos de la práctica	12
• Comprobar el comportamiento de JavaScript en modo estricto y en módulos.	12
• Identificar errores comunes por variables no declaradas.	12
• Aplicar estructuras de control para capturar posibles errores en funciones.....	12
• Documentar correctamente, como siempre , cada ejercicio con autoría y capturas de pantalla.....	12
• Presentar el proyecto en formato PDF junto con los archivos fuente y pantallazos. ..	12
CONCLUSIÓN	13

INTRODUCCIÓN

En el ejercicio “Probad todo el código del Apartado 6” deberemos incluir en uno o en varios fuentes todo el código que aparezca en la teoría del apartado correspondiente de JavaScript, así como su ejecución y autoría.

En el “Caso Práctico 6” ejecutaremos un programa en una etiqueta `<script>` dentro de un archivo HTML de las siguientes maneras que se explican en el ejercicio.

En el ejercicio “Ponte a prueba 6” deberemos modificar el código para que se capture el error que se producirá y muestre un mensaje por consola indicando el problema en cuestión. Si se produce un error la función devolverá -1.

ENUNCIADO

Probad todo el código del Apartado 6

Incluid en uno o varios fuentes **todo** el código que aparece en la teoría de ese apartado, así como su ejecución y prueba.

Ejemplo de implementación:

Captura de pantalla – DWEC_U02_A06_TEO_1.js

```
JS DWEC_U02_A06_TEO_1.js X
JS DWEC_U02_A06_TEO_1.js > ...
1 // Autor: Mario Segura Abad
2 // Fecha: 28/10/2025
3
4
5 // 6.1 Modo estricto
6 let miVariable = 1;
7
8 // ...Código del programa...
9
10 // Nombre de variable mal escrito (la 'v' es minúscula)
11 mivariable = 25; // No se genera error; además, se crea una variable nueva
12
13 "use strict";
14
15 let miVariable2 = 1;
16
17 //...Código del programa...
18
19 // Nombre de variable mal escrito
20 mivariable2 = 25; // ReferenceError: mivariable is not defined
21
22 function Coche(a,b) {
23     this.marca = a;
24     this.modelo = b;
25 }
26
27 let c1 = new Coche("Seat", "Ibiza");
28
29 console.log(c1.marca); // Seat
30 console.log(c1.modelo); // Ibiza
```

Captura de pantalla – DWEC_U02_A06_TEO_2.js

```
JS DWEC_U02_A06_TEO_2.js X
JS DWEC_U02_A06_TEO_2.js > ...
1 // Autor: Mario Segura Abad
2 // Fecha: 30/10/2025
3
4
5 // 6.2. ¿Clases u Objetos?
6 customElements.define('mi-componente',
7     class extends HTMLElement {
8         constructor() {
9             super();
10         }
11     }
12 );
```

Captura de pantalla – DWEC_U02_A06_TEO_3.js

```
JS DWEC_U02_A06_TEO_3.js X
JS DWEC_U02_A06_TEO_3.js > ...
1 // Autor: Mario Segura Abad
2 // Fecha: 30/10/2025
3
4
5 // 6.3. Manejo de errores
6 try {
7   // Acceso a una propiedad de una variable que no existe
8   console.log(hola.noexiste);
9 } catch (error) { // El parámetro puede tener cualquier nombre, no tiene por qué ser 'error'
10  console.log(error.name); // ReferenceError
11  console.log(error.message); // hola is not defined
12 }
13
14 try {
15   let a = "valornovalido";
16
17   if(a == "valornovalido") {
18     // Generamos un error con un mensaje personalizado
19     throw new Error("a no puede tener ese valor");
20   }
21 } catch (err) {
22   console.log(err.name); // Error
23   console.log(err.message); // a no puede tener ese valor
24 }
25
26 try {
27   // Apertura de conexión
28   // Procesamiento de datos
29 } catch (err) {
30   // Captura de errores
```

Caso práctico 6

Comprobación del modo estricto

Ejecuta el siguiente programa en una etiqueta **<script>** dentro de una página HTML de las siguientes tres maneras:

- De manera convencional.
- Activando el modo estricto.
- Sin modo estricto, pero activando el atributo **type="module"** en la etiqueta **<script>**.

Ejecuta el siguiente programa, con y sin modo estricto. ¿Sabrías explicar por qué no funciona en modo estricto?

En cada uno de los tres casos, responde a las siguientes preguntas:

- ¿El programa se ejecuta sin fallos?
- En caso afirmativo, ¿funciona correctamente de acuerdo con lo que se espera?
- ¿Puedes identificar los errores y explicar por qué se producen?

Ejemplo de implementación:

```
<> CasoPractico6.html X
Caso Práctico > <> CasoPractico6.html > ...
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 30/10/2025 -->
3
4  <!DOCTYPE html>
5  <html lang="es">
6      <head>
7          <meta charset="UTF-8">
8          <meta name="viewport" content="width=device-width, initial-scale=1.0">
9          <title>Caso Práctico 6 - Modo estricto en JavaScript</title>
10     </head>
11     <body>
12         <h1>Caso práctico 6: Comprobación del modo estricto</h1>
13
14         <p>Abre la consola del navegador (F12->Consola) para ver los resultados.</p>
15
16         <h2>1. Ejecución convencional (sin modo estricto)</h2>
17
18         <script>
19             function Enemigo(s, f) {
20                 salud = s;
21                 fuerza = f;
22             }
23
24             Enemigo.prototype.atacar = function() {
25                 console.log(`Atacando con fuerza ${fuerza}`);
26             }
27
28             let enemigo1 = new Enemigo(50, 20);
29             let enemigo2 = new Enemigo(40, 15);
30
31             // Ejecución de las acciones
32             enemigo1.atacar();
33             enemigo2.atacar();
34
35             // Fin de la ejecución
36
37             <!-- Prueba de Autoría -->
38             <p>Autor: <strong>Mario Segura Abad</strong></p><br>
39             <p>Fecha de realización: 30/10/2025</p>
40         </script>
41     </body>
42 </html>
```

```
function Enemigo(s, f) {
    salud = s;
    fuerza = f;
}
```

```
Enemigo.prototype.atacar = function() {
    console.log(`Atacando con fuerza ${fuerza}`);
}
```

```
let enemigo1 = new Enemigo(50, 20);
let enemigo2 = new Enemigo(40, 15);
```

```
enemigo1.atacar();
enemigo2.atacar();
```

Ponte a prueba 6

Enunciado

La siguiente función espera recibir como parámetro un objeto con una propiedad denominada **data**. Sin embargo, puedes comprobar que si se llama a la función sin parámetros o con **null** se producirá un error. Modifica el código para que se capture dicho error y se muestre un mensaje en la consola con el que se indique el problema. En caso de que produzca un error, la función deberá devolver **-1**.

Ejemplo de implementación:

```
PonteAPrueba.html X
Ponte a prueba > <> PonteAPrueba.html > html > body > script
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 31/10/2025 -->
3
4  <!DOCTYPE html>
5  <html lang="es">
6    <head>
7      <meta charset="UTF-8">
8      <meta name="viewport" content="width=device-width, initial-scale=1.0">
9      <title>Ponte a Prueba 6</title>
10   </head>
11   <body>
12     <h1>Ponte a prueba 6 - Manejo de errores en funciones</h1>
13     <p>Abre la consola del navegador (F12->Consola) para ver los resultados.</p>
14
15     <script>
16       function procesar(mensaje) {
17         try {
18           return mensaje.data;
19         } catch (error) {
20           console.error("Error: el parámetro 'mensaje' no es válido porque no tiene la propiedad 'data'.");
21           return -1;
22         }
23       }
24
25       // Pruebas
26       console.log("Caso 1: objeto con propiedad data");
27       console.log(procesar({data: "Hola mundo"})); // Devuelve "Hola mundo"
28
29       console.log("Caso 2: sin pasar parámetro");
30       console.log(procesar()); // Error --> devuelve -1
31
32       console.log("Caso 3: parámetro null");
33       console.log(procesar(null)); // Error --> devuelve -1
34
35       console.log("Caso 4: objeto sin propiedad data");
36       console.log(procesar({ mensaje: "Hola" })); // No lanza error, pero devuelve undefined
37     </script>
38
39     <!-- Prueba de Autoría -->
40     <p>Autor: <strong>Mario Segura Abad</strong></p><br>
41     <p>Fecha de realización: 31/10/2025</p>
42   </body>
43 </html>
```

```
function procesar(mensaje) {  
    return mensaje.data;  
}
```

Claves de resolución:

El acceso a **mensaje.data** provoca un error de acceso si **mensaje** es **null** o **undefined**. Por tanto, hay que englobar el código de la función en un bloque de control de errores para capturarlo. El bloque de procesamiento del error deberá realizar dos tareas: escribir el mensaje en la consola y devolver el valor **-1** como resultado de la función.

DESARROLLO DE LA PRACTICA

Práctica DWEC – Unidad 2

Descripción

Este proyecto se centra en la comprobación del comportamiento de JavaScript en distintos entornos de ejecución, especialmente en relación con el **modo estricto**. También se ha trabajado, como siempre, el **manejo de errores** mediante bloques try...catch, aplicando buenas prácticas para intentar evitar fallos en tiempo de ejecución cuando se accede a propiedades de objetos no definidos. El objetivo principal ha sido entender cómo cambia el comportamiento del lenguaje según el contexto de ejecución y cómo capturar errores de forma segura.

Contenido del archivo

Código del Apartado 6 de teoría

Se ha probado absolutamente todo el código de la teoría del apartado 6 de JavaScript en tres contextos dentro de una página HTML:

1. **De manera convencional:** sin modo estricto ni módulos.

Ejemplo de ejecución:

```
// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 28/10/2025");

undefined
undefined
undefined
undefined
Seat
Ibiza
Autor: Mario Segura Abad
Fecha de realización: 28/10/2025
< undefined
```

2. **Con modo estricto:** añadiendo “use strict” al inicio del script.

Ejemplo de ejecución:

```
// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 30/10/2025");

Autor: Mario Segura Abad
Fecha de realización: 30/10/2025
< undefined
```

3. **Como módulo:** usando type="module" en la etiqueta <script>. Ejemplo de ejecución:

```
// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 30/10/2025");
```

ReferenceError	VM1786:10
hola is not defined	VM1786:11
Error	VM1786:22
a no puede tener ese valor	VM1786:23
Autor: Mario Segura Abad	VM1786:37
Fecha de realización: 30/10/2025	VM1786:38

← undefined

Se ha documentado el comportamiento en cada caso, identificando errores y explicando por qué se producen. Además, se ha modificado una función que accede a propiedades de un objeto para que capture errores si el parámetro es null o undefined.

Caso práctico 6 – Comprobación del modo estricto

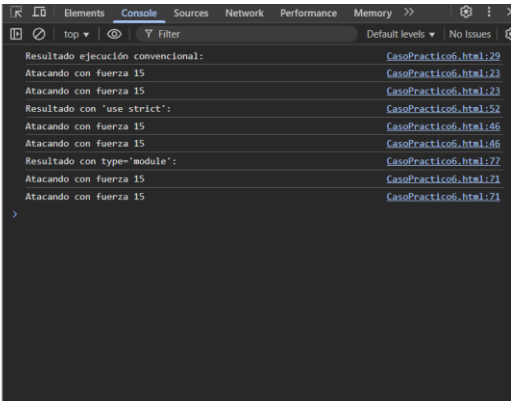
Ejemplo de ejecución:

Caso práctico 6: Comprobación del modo estricto

Abre la consola del navegador (F12->Consola) para ver los resultados.

- 1. Ejecución convencional (sin modo estricto)
- 2. Ejecución con modo estricto
- 3. Ejecución con type="module"

Autor: Mario Segura Abad
Fecha de realización: 30/10/2025



1. Ejecución convencional

- El programa se ejecuta sin fallos.
- Las variables salud y fuerza se crean de forma implícita en el ámbito global.
- Aunque funciona, este comportamiento es inseguro y propenso a errores.

Ejemplo de ejecución:

```
Resultado ejecución convencional:
2 Atacando con fuerza 15
```

2. Ejecución con modo estricto

- El programa lanza un error: ReferenceError al intentar asignar a salud y fuerza sin declararlas.
- Esto ocurre porque en modo estricto no se permite crear variables sin let, const o var.
- El error ayuda a detectar malas prácticas y obliga a declarar correctamente las variables.

Ejemplo de ejecución:

```
Resultado con 'use strict': CasoPractico6.html:52  
2 Atacando con fuerza 15 CasoPractico6.html:46
```

3. Ejecución como módulo (type="module")

- El comportamiento es similar al modo estricto.
- Se lanza el mismo error por no declarar las variables.
- Los módulos en JavaScript activan el modo estricto automáticamente.

Ejemplo de ejecución:

```
Resultado con type='module': CasoPractico6.html:77  
2 Atacando con fuerza 15 CasoPractico6.html:71  
Live reload enabled. CasoPractico6.html:116
```

Ponte a prueba – Control de errores con objetos

Se ha modificado la función procesar(mensaje) para evitar errores cuando el parámetro es null o no se proporciona.

Se ha utilizado un bloque try...catch para capturar el error y devolver -1 como valor seguro. Además, se muestra un mensaje en la consola con console.error() indicando el problema en cuestión.

Este ejercicio ha reforzado el uso de estructuras de control para evitar fallos inesperados y mejorar la robustez del código.

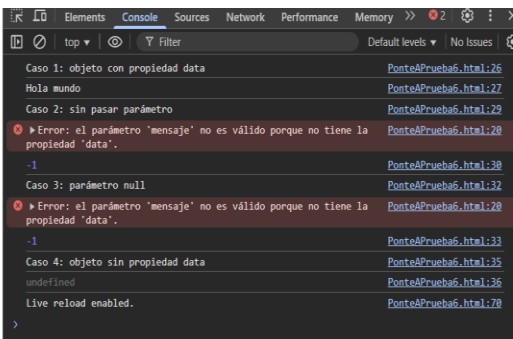
Ejemplo de ejecución:

Ponte a prueba 6 - Manejo de errores en funciones

Abre la consola del navegador (F12->Consola) para ver los resultados.

Autor: Mario Segura Abad

Fecha de realización: 31/10/2025



Cómo visualizarlo

Requisitos:

- Editor de código compatible: en mi caso, Visual Studio Code.
- Navegador web actual: en mi caso, Google Chrome.
- Extensión Live Server (opcional): para ejecutar y visualizar el resultado directamente en la pantalla del navegador pulsando en “Go Live”.

Pasos:

1. Guardar los archivos .html y .js en la raíz del proyecto, **muy importante este paso.**
2. Abrir el archivo .html en el navegador.
3. Probar cada versión del script (convencional, estricto, módulo).
4. Observar los resultados en la consola del navegador (F12->Consola).

Objetivos de la práctica

- Comprobar el comportamiento de JavaScript en modo estricto y en módulos.
- Identificar errores comunes por variables no declaradas.
- Aplicar estructuras de control para capturar posibles errores en funciones.
- Documentar correctamente, **como siempre**, cada ejercicio con autoría y capturas de pantalla.
- Presentar el proyecto en formato PDF junto con los archivos fuente y pantallazos.

CONCLUSIÓN

Esta práctica nos ha permitido entender cómo el **modo estricto** y los **módulos ES6** afectan al comportamiento del código de JavaScript.

Se ha comprobado que el modo estricto ayuda a detectar errores de declaración y mejora la seguridad del código. Además, se ha reforzado el uso de try...catch para capturar errores al acceder a propiedades de objetos no definidos, lo que mejora la estabilidad de las funciones.

La práctica, asimismo, ha sido de utilidad para aprender y consolidar conceptos de entorno de ejecución, buenas prácticas y control de errores con JavaScript.